Sistema de Tienda de Juegos Online

SISTEMA DE VENTA Y COMPRA DE JUEGOS DE FORMA ONLINE

Presentado Por:

Juan Pablo Mozuca Chaparro

Johan Sebastian Castro Gonzalez

David Santiago Beltran Pedraza

SERVICIO NACIONAL DE APRENDIZAJE “S.E.N.A”

CENTRO BIOTECNOLOGICO AGROPECUARIO TECNOLOGIA DE ANALISIS Y DESARROLLO DE SOFTWARE “A.D.S.O”

MOSQUERA CUNDINAMARCA, MARZO DE 2026

INDICE

1. Introducción 3
2. Casos de Uso – Cliente, Vendedor y Administrador 4
   2.1 Caso de Uso Logueo y Registro (Cliente y Vendedor) 4
   2.2 Caso de Uso de Reseña (Solo Cliente) 5
   2.3 Caso de Uso de Publicación de título (Solo vendedor) 6
   2.4 Casos de Uso de Retiro de Fondos (Solo vendedor) 7
   2.5 Caso de Uso de Moderación (Administrador) 8
   2.6 Caso de Uso de Gestión de Disputas 9
3. REQUERIMIENTOS TECNICOS 10
   - ✓ Almacenamiento: 250 GB HDD o SSD. 11
4. HERRAMIENTAS UTILIZADAS PARA EL DESARROLLO 11
   - ✓ Herramienta Técnica 1 – Lenguaje de programación del backend 11
   - ✓ Herramienta Técnica 2 – Framework de desarrollo web 11
   - ✓ Herramienta Técnica 3 – Implementación de API REST 11
   - ✓ Herramienta Técnica 4 – Autenticación de usuarios 11
   - ✓ Herramienta Técnica 5 – Base de datos 12
   - ✓ Herramienta Técnica 6 – Almacenamiento y gestión de imágenes 12
   - ✓ Herramienta Técnica 7 – Control de versiones 12
   - ✓ Herramienta Técnica 8 – Repositorio del proyecto 12
   - ✓ Herramienta Técnica 9 – Arquitectura del sistema 12
5. REQUERIMIENTOS 13
   5.1 REQUERIMIENTOS FUNCIONALES: 13
   - ✓ Requerimiento Funcional 1 – Registro de usuarios: 13
   - ✓ Requerimiento Funcional 2 – Inicio de sesión 13
   - ✓ Requerimiento Funcional 3 – Gestión de perfil de usuario 13
   - ✓ Requerimiento Funcional 4 – Catálogo de videojuegos 13
   - ✓ Requerimiento Funcional 5 – Búsqueda de videojuegos 14
   - ✓ Requerimiento Funcional 6 – Página de detalle del videojuego 14
   - ✓ Requerimiento Funcional 7 – Carrito de compras 14
   - ✓ Requerimiento Funcional 8 – Compra de videojuegos 14

- ✓ Requerimiento Funcional 9 – Biblioteca de juegos del usuario 14
- ✓ Requerimiento Funcional 10 – Administración de videojuegos (panel admin) 14
- ✓ Requerimiento Funcional 11 – Historial de compras..... 14
- ✓ Requerimiento Funcional 12 – Sistema de reseñas..... 15

5.2 REQUERIMIENTOS NO FUNCIONALES 15

- ✓ Requerimiento No Funcional 1 – Seguridad..... 15
- ✓ Requerimiento No Funcional 2 – Escalabilidad..... 15
- ✓ Requerimiento No Funcional 3 – Disponibilidad..... 15
- ✓ Requerimiento No Funcional 4 – Rendimiento..... 15
- ✓ Requerimiento No Funcional 5 – Compatibilidad 15
- ✓ Requerimiento No Funcional 6 – Seguridad de datos..... 15
- ✓ Requerimiento No Funcional 7 – Mantenibilidad..... 16
- ✓ Requerimiento No Funcional 8 – Usabilidad 16
- ✓ Requerimiento No Funcional 9 – Integridad de datos 16

6. Diseño de la Arquitectura..... 16

6.1 Roles 16

6.2 Dependencias 16

6.3 Rutas / EndPoints 17

6.4 Catalogo: 17

6.5 Reseñas: 17

6.6. Perfil: 17

6.7 Colecciones Firestore 17

- ✓ 6.7.1 Tabla de Juegos: 18
- ✓ 6.7.2 Tabla de Compras 18
- ✓ 6.7.3 Tabla de Registro 18
- ✓ 6.7.4 Tabla de Reseñas 18

7. Especificaciones de la API 19

8. Enlace del Repositorio 19

1. Introducción

El presente manual técnico describe PolyStation, una plataforma web de distribución digital de videojuegos diseñada para facilitar la gestión, comercialización y acceso a contenido interactivo en línea. Esta herramienta tecnológica permite a los usuarios explorar un catálogo de videojuegos, adquirir títulos de forma digital y administrar su biblioteca personal, todo dentro de un entorno seguro, eficiente y accesible desde cualquier dispositivo con conexión a internet.

En la actualidad, la industria de los videojuegos se ha consolidado como uno de los sectores más dinámicos del entretenimiento digital, donde la distribución en línea juega un papel fundamental en la disponibilidad inmediata de contenido y en la conexión entre desarrolladores y jugadores. Sin embargo, muchas plataformas requieren sistemas robustos que garanticen una adecuada gestión de usuarios, seguridad en los procesos de autenticación y un manejo eficiente de la información relacionada con videojuegos, compras y perfiles de usuario. Ante esta necesidad, PolyStation surge como una solución tecnológica que integra herramientas modernas de desarrollo web para ofrecer una experiencia digital confiable y escalable.

El sistema ha sido desarrollado utilizando tecnologías como Python y el framework Django para la lógica del backend, junto con servicios en la nube para la gestión de autenticación, almacenamiento de datos y manejo de recursos multimedia. Estas tecnologías permiten implementar un sistema modular, seguro y capaz de manejar múltiples solicitudes de usuarios de forma eficiente, garantizando el correcto funcionamiento de la plataforma.

Este manual tiene como propósito servir como guía técnica para la instalación, configuración y uso adecuado del sistema, proporcionando a desarrolladores, administradores y usuarios una referencia clara sobre su funcionamiento interno y sus principales componentes. De esta manera, se busca asegurar una correcta implementación de la plataforma, optimizar la gestión del catálogo de videojuegos y garantizar una experiencia de usuario fluida dentro del entorno digital de PolyStation.

2. Casos de Uso – Cliente, Vendedor y Administrador

2.1 Caso de Uso Logueo y Registro (Cliente y Vendedor)

Nombre:	Sistema de Tienda de Juegos Online "Polystation"
Descripción: Logueo o Registro de Usuario (Cliente/Vendedor)	

Caso de Uso:	Logueo o Registro de Usuario Externo		
Autor:		Fecha:	05/03/2026
Descripción: Proceso de autenticación para ingresar al sistema o creación de una nueva cuenta para gestionar compras o ventas.			
Actores:	Usuario Externo (Cliente o vendedor)		
Precondiciones			
El usuario debe contar con una conexión a internet y acceder a la URL principal de la plataforma Polystation. Flujo Normal			
Acción del Actor		Respuesta del Sistema	
1	El caso de uso inicia cuando el Usuario ingresa a la plataforma y selecciona la opción "Iniciar Sesión" o "Registrarse".	2	El sistema despliega el formulario correspondiente solicitando credenciales o datos de perfil.
3	En caso de registro, el Usuario selecciona su rol (comprador o vendedor) y acepta términos.	4	El sistema realiza el debido registro, crea el nuevo perfil y envía un correo de verificación.
5	El Usuario confirma sus datos o completa el inicio de sesión.	6	El sistema otorga el acceso y redirige al usuario al dashboard principal según el perfil seleccionado.
Flujo Alternativo			
2a	El Usuario ingresa una contraseña incorrecta o un correo no registrado.		El sistema muestra un mensaje de alerta: "Credenciales inválidas" y permite reintentar el ingreso.
Post Condiciones			
Frecuencia:		Tiempo esperado de Respuesta	

2.2 Caso de Uso de Reseña (Solo Cliente)

Nombre:	Sistema de Tienda de Juegos Online "Polystation"		
Descripción: El cliente califica con estrellas y comentarios un juego que ha comprado previamente.			
Caso de Uso:	Valoración y Reseña de Videojuego		
Autor:		Fecha:	05/03/2026
Actores:	Cliente		
Precondiciones			
El usuario debe haber adquirido el juego y haberlo ejecutado al menos una vez.			
Acción del Actor		Respuesta del Sistema	
1	El Cliente ingresa a su biblioteca de juegos y selecciona "Escribir Reseña".	2	El sistema despliega un formulario con escala de 1 a 5 estrellas y un campo de texto.
3	El Cliente redacta su opinión y califica el producto.	4	El sistema valida que el comentario no contenga palabras prohibidas y guarda la entrada.
5	El Cliente confirma el envío de la reseña.	6	El sistema actualiza el promedio de calificación del juego en la tienda pública.
Flujo Alternativo			
Post Condiciones			
Frecuencia:		Tiempo esperado de Respuesta	

2.3 Caso de Uso de Publicación de título (Solo vendedor)

Nombre:	Sistema de Tienda de Juegos Online "Polystation"		
Descripción: El vendedor registra un nuevo videojuego en el catálogo de la tienda.			
Caso de Uso:	Publicación de Nuevo Título (Listing)		
Autor:		Fecha:	05/03/2026
Actores:	Vendedor		
Precondiciones			
El usuario debe tener una cuenta de vendedor activa y verificada.			
Acción del Actor		Respuesta del Sistema	
1	El Vendedor selecciona "Añadir Producto" en su panel de control.	2	El sistema muestra un formulario de registro (Título, Género, Precio, Stock).
3	El Vendedor sube las imágenes del juego y pega las llaves (keys) de activación.	4	El sistema valida el formato de las imágenes y la estructura de los códigos.
5	El Vendedor hace clic en "Publicar para Revisión".	6	El sistema guarda el producto en estado "Pendiente" y notifica al administrador.
Flujo Alternativo			
Post Condiciones			
Frecuencia:		Tiempo esperado de Respuesta	

2.4 Casos de Uso de Retiro de Fondos (Solo vendedor)

Nombre:	Sistema de Tienda de Juegos Online "Polystation"		
Descripción: El vendedor solicita transferir el saldo acumulado por sus ventas a su cuenta bancaria.			
Caso de Uso:	Retiro de Fondos y Liquidación		
Autor:		Fecha:	05/03/2026
Actores:	Vendedor		
Precondiciones			
El saldo disponible debe ser superior al monto mínimo de retiro.			
Acción del Actor		Respuesta del Sistema	
1	El Vendedor ingresa a la sección "Billetera" y selecciona "Retirar Fondos".	2	El sistema muestra el saldo total y los métodos de retiro configurados.
3	El Vendedor ingresa el monto a retirar y confirma su contraseña.	4	El sistema descuenta el saldo temporalmente y genera una solicitud de pago.
5	El Vendedor recibe un comprobante de transacción pendiente.	6	El sistema envía una alerta al departamento financiero para procesar el pago.
Flujo Alternativo			
Post Condiciones			
Frecuencia:		Tiempo esperado de Respuesta	

2.5 Caso de Uso de Moderación (Administrador)

Nombre:	Sistema de Tienda de Juegos Online "Polystation"		
Descripción: El administrador aprueba o rechaza los juegos publicados por los vendedores.			
Caso de Uso:	Moderación de Contenido y Productos		
Autor:		Fecha:	05/03/2026
Actores:	Administrador		
Precondiciones			
Deben existir productos en estado "Pendiente de Revisión".			
Acción del Actor		Respuesta del Sistema	
1	El Administrador accede a la cola de moderación.	2	El sistema lista todos los productos nuevos subidos por los vendedores.
3	El Administrador revisa que la descripción y los precios sean correctos.	4	El sistema permite marcar el producto como "Aprobado" o "Rechazado".
5	El Administrador confirma la decisión.	6	El sistema publica el juego en la tienda (si es aprobado) o notifica al vendedor el motivo del rechazo.
Flujo Alternativo			
Post Condiciones			
Frecuencia:		Tiempo esperado de Respuesta	

2.6 Caso de Uso de Gestión de Disputas

Nombre:	Sistema de Tienda de Juegos Online "Polystation"		
Descripción: El administrador actúa como mediador cuando un cliente reporta una clave fallida.			
Caso de Uso:	Gestión de Disputas (Soporte Técnico)		
Autor:		Fecha:	05/03/2026
Actores:	Administrador, Vendedor, Cliente		
Precondiciones			
El cliente debe haber abierto un ticket de reclamo por una compra.			
Acción del Actor		Respuesta del Sistema	
1	El Administrador abre el ticket de disputa en el panel de control.	2	El sistema muestra el chat entre cliente/vendedor y la evidencia subida.
3	El Administrador verifica la validez de la clave con el proveedor.	4	El sistema habilita botones de acción: "Reembolsar" o "Cerrar Disputa".
5	El Administrador decide a favor del cliente y ejecuta el reembolso.	6	El sistema devuelve el dinero al cliente y penaliza la reputación del vendedor.
Flujo Alternativo			
Post Condiciones			
Frecuencia:		Tiempo esperado de Respuesta	

3. REQUERIMIENTOS TECNICOS

REQUERIMIENTOS MINIMOS DE HARDWARE

Servidor de Aplicaciones y Base de Datos

- ✓ Procesador: Intel Core i5 (8ª generación o superior) o equivalente AMD Ryzen 5.
- ✓ Memoria RAM: 8 GB (mínimo) / 16 GB (recomendado).
- ✓ Almacenamiento: 500 GB HDD (mínimo) / 256 GB SSD (recomendado para mejor rendimiento).
- ✓ Red: Conexión Ethernet de 1 Gbps.
- ✓ Monitor: Resolución mínima de 1366x768 px.

Estaciones de Trabajo (Operadores/Analistas)

- ✓ Procesador: Intel Core i3 (7ª generación o superior) o equivalente AMD Ryzen
- ✓ Memoria RAM: 4 GB (mínimo) / 8 GB (recomendado).
- ✓ Almacenamiento: 250 GB HDD o SSD.
- ✓ Conexión de red: WiFi 802.11n o Ethernet.
- ✓ Monitor: Resolución mínima de 1366x768 px.

4. HERRAMIENTAS UTILIZADAS PARA EL DESARROLLO

- ✓ Herramienta Técnica 1 – Lenguaje de programación del backend

El backend del sistema debe desarrollarse utilizando el lenguaje de programación **Python**, debido a su amplia compatibilidad con frameworks web, bibliotecas para APIs y facilidad de integración con servicios externos.

- ✓ Herramienta Técnica 2 – Framework de desarrollo web

El sistema debe utilizar el framework **Django** para la construcción del backend, permitiendo una arquitectura robusta, modular y segura para el desarrollo de aplicaciones web.

- ✓ Herramienta Técnica 3 – Implementación de API REST

La comunicación entre el frontend y el backend debe realizarse mediante una API REST construida con **Django REST Framework**, permitiendo el manejo eficiente de solicitudes HTTP, serialización de datos y control de permisos.

- ✓ Herramienta Técnica 4 – Autenticación de usuarios

El sistema debe implementar autenticación y gestión de usuarios utilizando **Firebase Authentication**, permitiendo manejar registro, inicio de sesión y verificación de identidad mediante tokens seguros.

- ✓ Herramienta Técnica 5 – Base de datos

El almacenamiento de la información de la plataforma (usuarios, videojuegos, compras, reseñas, etc.) debe realizarse utilizando **Cloud Firestore**, una base de datos NoSQL que permite almacenamiento en tiempo real, escalabilidad automática y sincronización eficiente con aplicaciones web.

- ✓ Herramienta Técnica 6 – Almacenamiento y gestión de imágenes

Las imágenes de los videojuegos (portadas, capturas de pantalla, banners, etc.) deben almacenarse y gestionarse mediante **Cloudinary**.

Este servicio permitirá:

- Almacenamiento en la nube de imágenes y recursos multimedia.
- Optimización automática de imágenes para mejorar el rendimiento de la plataforma.
- Transformación dinámica de imágenes (redimensionamiento, compresión y formatos).
- Entrega rápida de contenido mediante CDN para mejorar los tiempos de carga.

✓ Herramienta Técnica 7 – Control de versiones

El desarrollo del proyecto debe gestionarse mediante **Git**, permitiendo el seguimiento de cambios en el código, trabajo colaborativo entre desarrolladores y control de versiones del sistema.

✓ Herramienta Técnica 8 – Repositorio del proyecto

El código fuente del sistema debe almacenarse en un repositorio remoto utilizando **GitHub** o una plataforma similar, facilitando la colaboración entre desarrolladores y la gestión del proyecto.

✓ Herramienta Técnica 9 – Arquitectura del sistema

El sistema debe seguir una arquitectura **cliente-servidor**, donde:

- El **backend** se encargará de la lógica de negocio y gestión de datos.
- El **frontend** consumirá los servicios del backend mediante API REST.
- Los servicios externos (Firebase y Cloudinary) se integrarán para autenticación, almacenamiento de datos y manejo de recursos multimedia.

5. REQUERIMIENTOS

Los requerimientos que solicitamos para la creación del aplicativo de la Plataforma distribuidora digital de videojuegos "PolyStation" para los que debemos tener claro **qué hará la plataforma** y **cómo funcionará técnicamente, en este caso serán los siguientes**:

5.1 REQUERIMIENTOS FUNCIONALES:

Los **requerimientos funcionales que mostraran que funciones tendrá nuestro sistema serán los siguientes:**

✓ Requerimiento Funcional 1 – Registro de usuarios:

El sistema debe permitir que los usuarios creen una cuenta utilizando correo electrónico y contraseña o métodos de verificación con otros servicios(inicio de sesión con Google, Facebook por ejemplo) mediante **Firebase Authentication**.

✓ Requerimiento Funcional 2 – Inicio de sesión

El sistema debe permitir a los usuarios autenticarse utilizando correo y contraseña mediante Firebase Authentication.

✓ Requerimiento Funcional 3 – Gestión de perfil de usuario

El sistema debe permitir a los usuarios:

- Ver su perfil.
- Editar información básica (nombre, foto, país, etc.).
- Cambiar contraseña.

✓ Requerimiento Funcional 4 – Catálogo de videojuegos

El sistema debe mostrar un catálogo de videojuegos disponibles con información como:

- Nombre
- Descripción
- Precio
- Categoría
- Desarrollador
- Imágenes o portada
- Requisitos del sistema

✓ Requerimiento Funcional 5 – Búsqueda de videojuegos

El sistema debe permitir buscar videojuegos por:

- Nombre
- Categoría
- Género
- Precio

✓ Requerimiento Funcional 6 – Página de detalle del videojuego

El sistema debe mostrar la información completa de cada videojuego seleccionado.

✓ Requerimiento Funcional 7 – Carrito de compras

El sistema debe permitir a los usuarios:

- Agregar videojuegos al carrito
- Eliminar videojuegos del carrito
- Visualizar el total de la compra

✓ Requerimiento Funcional 8 – Compra de videojuegos

El sistema debe permitir a los usuarios comprar videojuegos seleccionados.

✓ Requerimiento Funcional 9 – Biblioteca de juegos del usuario

El sistema debe permitir a los usuarios ver los videojuegos que han comprado.

✓ Requerimiento Funcional 10 – Administración de videojuegos (panel admin)

El sistema debe permitir a los administradores:

- Agregar videojuegos
- Editar videojuegos
- Eliminar videojuegos
- Subir imágenes o archivos asociados

✓ Requerimiento Funcional 11 – Historial de compras

El sistema debe permitir a los usuarios consultar el historial de compras realizadas.

✓ Requerimiento Funcional 12 – Sistema de reseñas

El sistema debe permitir a los usuarios:

- Calificar videojuegos
- Escribir reseñas

5.2 REQUERIMIENTOS NO FUNCIONALES

En los requerimientos no funcionales que mostrarán como deberá y va a comportarse nuestr**a plataforma,** en este caso en concreto lo que vamos a buscar será lo siguiente:

✓ Requerimiento No Funcional 1 – Seguridad

El sistema debe implementar autenticación segura mediante **Firebase Authentication** y protección de endpoints en Django mediante tokens.

✓ Requerimiento No Funcional 2 – Escalabilidad

El sistema debe permitir escalar el almacenamiento de datos utilizando **Firestore** para manejar grandes cantidades de usuarios y videojuegos.

✓ Requerimiento No Funcional 3 – Disponibilidad

La plataforma debe tener una disponibilidad mínima del **99% del tiempo**.

✓ Requerimiento No Funcional 4 – Rendimiento

Las consultas al catálogo de videojuegos deben responder en menos de **2 segundos** bajo condiciones normales.

✓ Requerimiento No Funcional 5 – Compatibilidad

La plataforma web debe ser compatible con los navegadores:

- Chrome
- Firefox
- Edge
- Safari

✓ Requerimiento No Funcional 6 – Seguridad de datos

La información de los usuarios debe almacenarse de forma segura y cumplir prácticas de protección de datos.

✓ Requerimiento No Funcional 7 – Mantenibilidad

El backend debe desarrollarse siguiendo la arquitectura de **Django REST API** para facilitar mantenimiento y escalabilidad.

✓ Requerimiento No Funcional 8 – Usabilidad

La interfaz debe permitir que un usuario nuevo pueda comprar un videojuego en **menos de 5 pasos**.

✓ Requerimiento No Funcional 9 – Integridad de datos

Las transacciones de compra deben garantizar que el usuario solo pueda acceder a juegos que haya comprado.

- ✓ El sistema debe registrar en bitácoras (logs) todas las acciones críticas de los usuarios, permitiendo auditorías internas y externas.

6. Diseño de la Arquitectura

6.1 Roles

-Jugador: que como tal el es el que realiza una compra, puede hacer reseñas y cuenta con una biblioteca propia

-Creador: Es el que publica videojuegos, este mismo los gestiona además de subir portadas contando con el servicio de cloudinary

-Admin: Cuenta con acceso total a la pagina, gestiona los usuarios y los roles

6.2 Dependencias

Django

Drf-Spectacular

Firebase Admin

Cloudinary

DotEnv

6.3 Rutas / EndPoints

En el caso de las endpoints de la api contamos con varias

Autenticación:

Contamos con 2 de método POST que vienen con esta ruta /auth/registro/ y /auth/login/

6.4 Catalogo:

En el catalogo tenemos 2 rutas, una en método GET, una en método POST que serian /juegos/ y /juegos/

en el caso de POST seria solo para el rol de creador, en caso de publicar un juego

6.5 Reseñas:

Tenemos 2 rutas, una en método POST y otra en método GET, la url es /resenas/

En este caso la misma ruta aplica para los dos métodos

6.6. Perfil:

Que contamos con un método Post que en este caso seria para subir la foto de perfil usando cloudinary, la ruta es

/perfil/foto/

6.7 Colecciones Firestore

Toda la información se almacenaría en Cloud Firestore, estaría organizada en las siguientes colecciones

- ✓ 6.7.1 Tabla de Juegos:

CAMPO	TIPO	DESCRIPCION
titulo	string	Título del videojuego
descripcion	string	Descripción detallada del juego
precio	number	Precio en la tienda
genero	string	Género del juego (acción, RPG, etc)
plataforma	string	Plataformas compatibles
portada_url	string	URL segura de Cloudinary
creador_id	string	uid del creador dueño del juego
fecha_publicacion	timestamp	Generado automáticamente con SERVER_TIMESTAMP

- ✓ 6.7.2 Tabla de Compras

CAMPO	TIPO	DESCRIPCION
jugador_id	string	uid del jugador que realizó la compra
juego_id	string	ID del documento del juego comprado
precio_pagado	number	Precio en el momento de la compra
fecha_compra	timestamp	Generado automáticamente con SERVER_TIMESTAMP

- ✓ 6.7.3 Tabla de Registro

CAMPO	TIPO	DESCRIPCION
email	string	Correo electrónico del usuario
nombre	string	Nombre del usuario
rol	string	jugador | creador | admin
foto_url	string	URL de foto de perfil en Cloudinary
activo	boolean	Estado de la cuenta (gestionado por Admin)
fecha_registro	timestamp	Generado automáticamente con SERVER_TIMESTAMP

- ✓ 6.7.4 Tabla de Reseñas

CAMPO	TIPO	DESCRIPCION
jugador_id	string	uid del autor de la reseña
juego_id	string	ID del juego reseñado
puntaje	number	Calificación del 1 al 5
comentario	string	Texto de la reseña
fecha	timestamp	Generado automáticamente con SERVER_TIMESTAMP

7. Especificaciones de la API

8. Enlace del Repositorio

Enlace: https://github.com/PabloMSL/PolyStation